12_23_aprile

Memoria Virtuale e Paginazione

La **Memoria Virtuale** è un'astrazione: dà l'idea a noi e al processo che l'allocazione sia contigua, in realtà il sistema operativo spezzetta l'allocazione di RAM per quel processo.

Questo meccanismo è la **paginazione**, che fa lavorare il processo in una memoria virtuale con indirizzi da 0 a un tetto massimo virtuale. Questo elimina l'esigenza di avere una memoria massima o espandibile. In realtà, il pezzo di memoria dedicata al processo è composto da tante pagine della stessa dimensione.

I vari processi poi condividono la stessa RAM per la loro memoria virtuale; anche se la RAM è piccola rispetto alle varie pagine, non cambia poiché le memorie sono virtuali.

La **memoria fisica** è divisa in blocchi della stessa dimensione della pagina (**Frame**) per facilitare l'assemblaggio della memoria virtuale.

- **Vantaggio:** Questo risolve la frammentazione esterna, perché si andranno ad evitare buchi nella RAM dato che il SO lo riempirà assegnandolo ad un processo.
- **Svantaggio:** Non risolve la frammentazione interna (più la pagina è grande, più ci sarà frammentazione interna).

Per ogni processo tengo in RAM solo alcune pagine, poiché il Sistema Operativo gestisce cosa a un determinato processo non serve in RAM, e il resto sta in disco.

> ⓘ **Page Fault**
>
> Avviene quando un processo richiede l'uso di un qualcosa (strutture dati, ecc.) che attualmente si trova in disco e non in RAM. Il processo non sa che parte delle sue pagine sono in disco; se accade un *page fault* si genera un'**eccezione** per spostare ciò che serve in RAM.

Nelle memorie virtuali abbiamo gli **indirizzi virtuali**. Questi indirizzi vengono mappati in memoria fisica. Ogni processo ha il proprio indirizzo virtuale che punta a un luogo diverso: avrò una **Tabella delle Pagine** per ogni processo che ne segna la mappatura in RAM.

- Grazie a questo metodo si può estrarre l'indirizzo fisico istantaneamente.
- Il **Bit di Presenza** serve a vedere se la pagina è o no in RAM (o in disco). Se è in RAM, devo specificare in che *frame* si trova questa pagina.

La CPU genera un indirizzo virtuale che deve essere tradotto in indirizzo fisico dalla **MMU** (Memory Management Unit).

Il ruolo della MMU nella Traduzione

L'MMU per tradurre esegue una divisione:

$$\frac{\text{Indirizzo}_V}{\text{Dim}} = \begin{cases} Q \rightarrow \text{Numero di Pagina} \\ R \rightarrow \text{Offset} \end{cases}$$

Esempio:

Dati $I_V = 8196$ e $\text{Dim} = 4096$:

$$\frac{8196}{4096} = \begin{cases} 2 \text{ (Numero di Pagina)} \\ 4 \text{ (Offset)} \end{cases}$$

Definizione di Offset

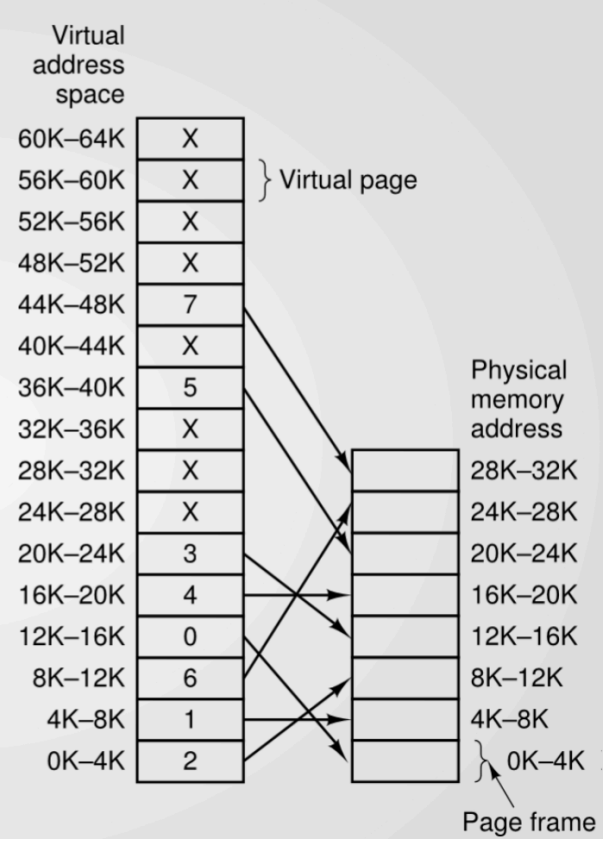
Distanza dall'inizio della pagina alla *word* che sto referenziando.

Dalla Tabella delle Pagine, associo il Numero di Pagina trovato al Frame. Dopodiché calcolo l'indirizzo fisico:

$$N_{FRAME} \cdot \text{DIM} + \text{OFFSET} = \text{Indirizzo Fisico}$$

Tabella delle Pagine

E' una per processo. La tabella delle pagine è contenuta in RAM, e gli aspetti da curare sono 2: la velocità di consultazione e la dimensione



- **Indice** K: per ogni pagina ho un *Bit di presenza* e un *Numero di frame*.
- **Numero di Pagina**: Saranno i bit più significativi dati da $M - m$, dove:
  - M = esponente dell'indirizzo virtuale (2^M)
  - m = esponente della dimensione della pagina (2^m)
- **Offset**: Saranno gli m bit meno significativi.

Si può generalizzare con 2^{M-m} frame facendo uno shift.

Esercizio d'Esame

Input:

- Memoria Virtuale = 4 MB
- Tabella Pagine = 2^{13} Voci $\implies$ Dim. Pagina Fisica = 13 bit
- Numero Frame = 8 bit

Richiesta:

Dimensione Memoria Fisica (RAM)?

Svolgimento:

Sappiamo che:

$$4 \text{ MB} = 2^{22} \text{ byte} \implies \text{Dimensione Indirizzo Virtuale} = 22 \text{ bit}$$

Calcolo dell'Offset:

$$\text{Offset} = 22 - 13 = 9 \text{ bit} \implies \text{Dimensione Pagina} = 2^9 \text{ byte}$$

Calcolo della memoria fisica (RAM):

$$2^8 \text{ (N. Frame)} \cdot 2^9 \text{ (Dim. Pagina)} = 2^{17} \text{ byte} = 128 \text{ KB} \implies \text{RAM}$$

Oppure, calcolo alternativo per la dimensione della pagina:

$$\frac{2^{22}}{2^{13}} = 2^9 \text{ byte}$$

Tabella delle Pagine e PTE (Page Table Entry)

Page Table Entry (PTE)

Ogni riga della tabella delle pagine è chiamata **PTE**. È il "ponte" che permette al processore di capire a quale indirizzo fisico corrisponda un indirizzo virtuale generato dal programma. Contiene una serie di campi e informazioni fondamentali gestiti in sinergia tra MMU e Sistema Operativo (SO).

I Campi di una PTE

1. Numero del Frame (Indirizzo Fisico)

Contiene l'indirizzo fisico di base. Quando la CPU accede a un indirizzo virtuale, prende questo numero e lo concatena all'offset per calcolare l'indirizzo reale in RAM in cui si trova il dato.

2. Bit Presente / Assente

Indica se la pagina si trova attualmente in RAM (Frame) oppure se risiede solo su disco. Se assente, un accesso genererà un *Page Fault*.

3. Bit di Protezione

Definisce i diritti di accesso per quella specifica pagina: **Lettura**, **Scrittura** ed eventualmente **Esecuzione**.

- **Protezione del Codice:** Lo spazio di memoria virtuale è diviso in *Stack* (in alto, cresce verso il basso), *Heap* (al centro, cresce verso l'alto) e l'*Area Dati/Codice* (in basso). Spesso l'area del codice è di sola lettura. Dato che il codice non si automodifica (sarebbe più una complicazione che un vantaggio), più istanze dello stesso programma possono condividere in memoria la stessa area di codice.

- **Perché risiede nella Tabella delle Pagine?** Perché la tabella è letta direttamente dalla **MMU**. La MMU fa da "guardia": se rileva un'operazione non consentita (es. scrittura su codice), blocca tutto e segnala un'eccezione al SO (che di norma termina il processo).
- **Prevenzione attacchi:** Evita tecniche di *Code Injection*, in cui si cerca di inserire codice malevolo in aree destinate ai dati (come Heap o Stack) per poi eseguirlo o per mandare in crash l'applicazione.

4. Bit di Modifica (Dirty Bit)

Segnala se la pagina in RAM è stata modificata ("sporca") rispetto alla sua copia originale salvata su disco.

- **Flusso con Page Fault:**
  1. La CPU cerca una pagina non in RAM → si verifica un *Page Fault*.
  2. La pagina viene copiata dal disco alla RAM. La copia su disco non viene cancellata.
  3. All'inizio il Dirty Bit è a **0** (pagina pulita, identica alla copia su disco).
  4. Se il processo effettua una scrittura su quella pagina, la MMU aggiorna il Dirty Bit a **1**.
- **Gestione della "Pagina Vittima":** Quando la RAM è piena e il SO deve fare spazio spostando pagine su disco, usa questo bit per ottimizzare le operazioni:
  - *Dirty Bit = 0:* Il SO sovrascrive semplicemente il frame in RAM senza copiare nulla su disco (risparmiando un'operazione di I/O molto lenta).
  - *Dirty Bit = 1:* Il SO è obbligato a salvare le modifiche scrivendo la pagina su disco prima di liberare il frame.

> ⓘ **A volte il SO approfitta dei momenti in cui non ha operazioni pendenti (inattività della CPU) per sincronizzare le pagine sporche su disco, riportando preventivamente il Dirty Bit a 0 per velocizzare future sostituzioni.**

5. Bit di Referenziamento (Bit R)

Viene impostato a **1** dalla MMU in automatico ogni volta che avviene un accesso alla pagina (sia in lettura che in scrittura).

- **Utilità:** Fornisce una sorta di *microstatistica* sull'uso delle pagine, a costo quasi nullo per il SO.
- **Come funziona:** Il SO azzera periodicamente questi bit (es. ogni 500 ms). Se alla fine del periodo il bit è a 1, significa che il processo ha usato la pagina recentemente. Monitorando questi cicli, il SO capisce quali pagine sono usate attivamente e quali possono essere scartate in caso di necessità.

6. Bit per Disabilitare la Cache

Segnala che la linea di cache corrispondente a quella pagina non deve essere utilizzata, forzando la lettura direttamente dalla RAM (o dalla periferica).

- **Il problema dell'I/O (Memory-Mapped I/O):** L'hardware mappa alcune porte delle periferiche I/O su specifici indirizzi di memoria, permettendo ai processi di leggere lo stato di un dispositivo tramite un normale "fetch" di memoria.
- **Perché disabilitare la cache?** Se la cache fosse attiva su questi indirizzi speciali, restituirebbe un valore obsoleto salvato in cache, impedendo la lettura in tempo reale dello stato effettivo della porta I/O.

7. Bit di Validità / Allocazione

Traccia se una determinata area dello spazio di indirizzamento è stata effettivamente allocata dal SO per il processo.

- **Espansione dinamica:** Anche se le pagine tra Heap e Stack sembrano libere e appartenenti al processo, per usarle (es. allocando vettori con `malloc`) il processo deve chiedere al SO di spostare il limite dell'Heap ("più in alto").
- **Ottimizzazione e Sicurezza:** Il SO deve sapere esattamente quali pagine sono vuote e quali allocate per ottimizzare le dimensioni delle tabelle delle pagine (che altrimenti su sistemi a 64 bit avrebbero dimensioni spropositate, es. 2^{52} voci).
- **Segmentation Fault:** Se la MMU rileva un accesso a un indirizzo non allocato dal SO (es. area mai richiesta dal processo), genera un'eccezione come il *Segmentation Fault*, prevenendo comportamenti imprevisti.

ⓘ Differenza tra FRAME e PAGINA

Un Frame corrisponde esattamente ad una pagina, solo che il frame contiene gli indirizzi fisici presenti in RAM, mentre la pagina contiene gli indirizzi logici che vede il processo. Dal punto di vista del processo lui sta lavorando su indirizzi di memoria fisici tutti contigui, ma in realtà ne ha solo l'impressione perché gli indirizzi fisici reali li conosce solo il SO, questo viene fatto per evitare il problema della frammentazione esterna